Project 3 — Problem 1

EECE 5614: Reinforcement Learning and Decision Making Under Uncertainty

Shoukang Yu
Northeastern University

Spring 2026

1 Problem Setup and Parameters

We revisit the 20×20 stochastic maze from Project 2, now solved with three temporal difference (TD) learning algorithms: **Q-Learning**, **SARSA**, and **Tabular Actor-Critic**. Unlike the matrix-form dynamic programming methods in Project 2, these algorithms learn directly from sampled experience without requiring explicit knowledge of the transition model.

Parameter	Value
Stochastic drift p	0.025
Discount factor γ	0.96
Learning rate α	0.25
Exploration rate ϵ	0.1 (Q-Learning, SARSA)
Actor learning rate β	0.05 (default); 0.2 (Actor-Critic tuned)
Number of episodes	1,000 (Q-Learning, SARSA); 2,000 (Actor-Critic tuned)
Max steps per episode	1,000
Number of independent runs	10

Table 1: Hyperparameters for Problem 1.

The three algorithms differ in their update rules:

- **Q-Learning** (off-policy): $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$
- **SARSA** (on-policy): $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma Q(s',a') - Q(s,a)\big]$
- **Actor-Critic**: The critic updates a state-value function $V(s)$ with TD error $\delta = r + \gamma V(s') - V(s)$, while the actor updates action preferences $H(s,a)$ via $H(s,a) \leftarrow H(s,a) + \beta\,\delta\,(1 - \pi(a|s))$, where $\pi(a|s) = \text{softmax}(H(s,\cdot))$.

2 Q1: Path Success Rate

After training, we execute each algorithm's learned greedy policy from the start state and check whether the goal is reached within 1,000 steps.

Algorithm	Paths Found (out of 10)
Q-Learning	**10** / 10
SARSA	**10** / 10
Actor-Critic	**2** / 10

Table 2: Number of successful runs for each algorithm.

Note: Actor-Critic results are reported after parameter tuning ($\beta = 0.2$, 2,000 episodes). With default parameters ($\beta = 0.05$, 1,000 episodes), Actor-Critic achieved 0/10.

Key code:

```
# Q-Learning update: off-policy (uses max over next actions)
Q[state, action] += alpha * (  # Q[state, action]: Q(s,a) ← Q(s,a) + α [r + γ max_a' Q(s',a') - Q(s,a)]
    reward + gamma * np.max(Q[next_state]) - Q[state, action]
)
```

Figure 1: Q-Learning core update implementation.

```
# SARSA update: on-policy (uses next action actually taken)
Q[state, action] += alpha * (
    reward + gamma * Q[next_state, next_action] - Q[state, action]  # Q(s,a) ← Q(s,a) + α [r + γ Q(s',a') - Q(s,a)]
)
```

Figure 2: SARSA core update implementation.

```
# TD error
delta = reward + gamma * V[next_state] - V[state]

# Critic update
V[state] += alpha * delta

# Actor update: increase preference for good actions
H[state, action] += beta * delta * (1.0 - pi[action])
```

Figure 3: Actor-Critic core update implementation.

Discussion: Q-Learning achieves a 10/10 success rate because its off-policy update $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$ always bootstraps toward the greedy maximum, propagating the goal reward backward through the state space aggressively. SARSA also achieves 10/10 but its on-policy target $r + \gamma Q(s',a')$ is averaged over exploratory ε-greedy actions, yielding slightly slower but equally reliable convergence. Actor-Critic achieves only 2/10 because the actor update $H(s,a) \leftarrow H(s,a) + \beta\,\delta\,(1 - \pi(a|s))$ depends on the accuracy of the TD error $\delta = r + \gamma V(s') - V(s)$: when V is poorly initialized, δ is noisy and the policy gradient step drives H in the wrong direction, creating a chicken-and-egg dependency between critic accuracy and actor improvement that makes convergence rare within 2,000 episodes.

3 Q2: Optimal Policy and Optimal Path

For each algorithm, we select one successful run and display the learned greedy policy (arrows indicating the best action at each state) and a sampled trajectory from start to goal under that policy.

3.1 Q-Learning

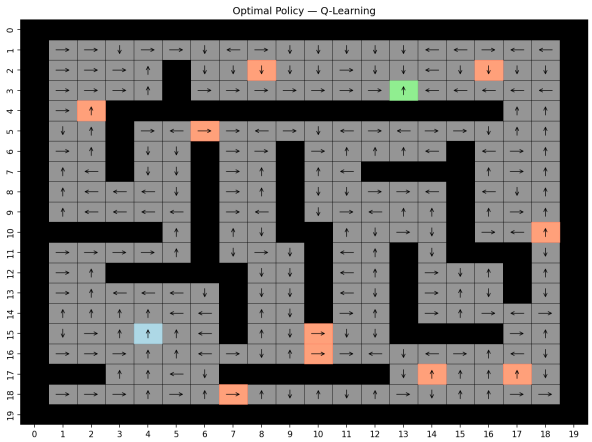

Figure 4: Optimal policy — Q-Learning.

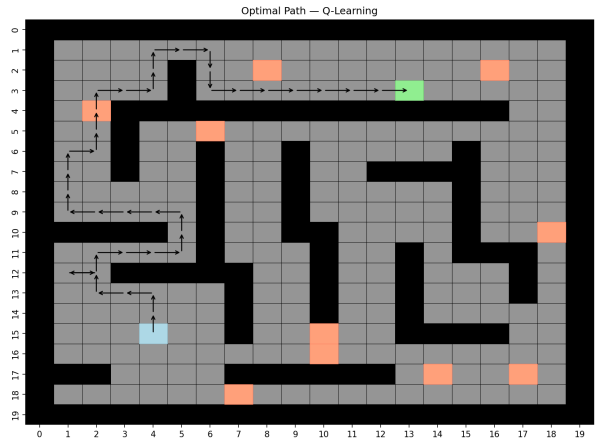

Figure 5: Optimal path — Q-Learning.

Discussion: The learned policy (Figure 4) assigns consistent directional arrows throughout the maze, with states near the goal pointing upward toward state 208. The greedy path (Figure 5) successfully navigates from the start (blue) to the goal (green), taking a route that avoids walls and penalty cells. The path shows occasional detours due to stochastic transitions ($p = 0.025$), but the overall direction remains correct, confirming that Q-Learning has learned a reliable value function.

3.2 SARSA

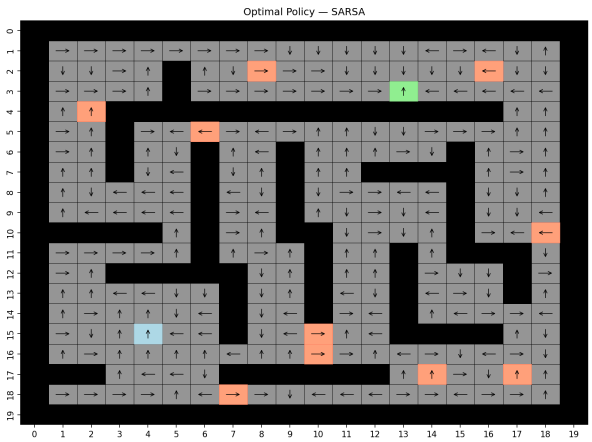

Figure 6: Optimal policy — SARSA.

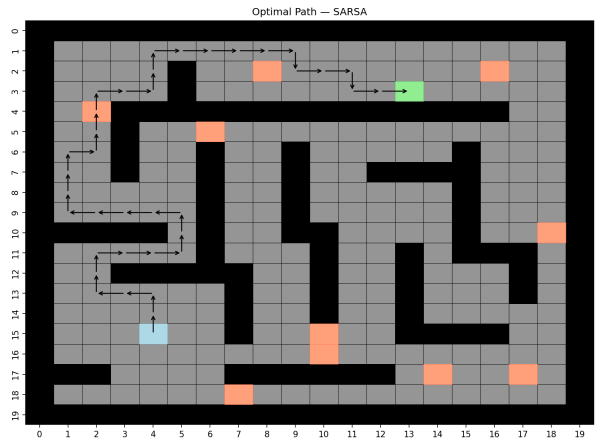

Figure 7: Optimal path — SARSA.

Discussion: The SARSA policy (Figure 6) is visually similar to Q-Learning's, with directional arrows guiding the agent toward the goal. However, because SARSA is on-policy, its Q-values reflect the actual ε-greedy behavior during training rather than the optimal greedy policy, resulting in slightly more conservative estimates near walls. The resulting path (Figure 7) reaches the goal successfully, with a trajectory comparable to Q-Learning.

3.3 Actor-Critic

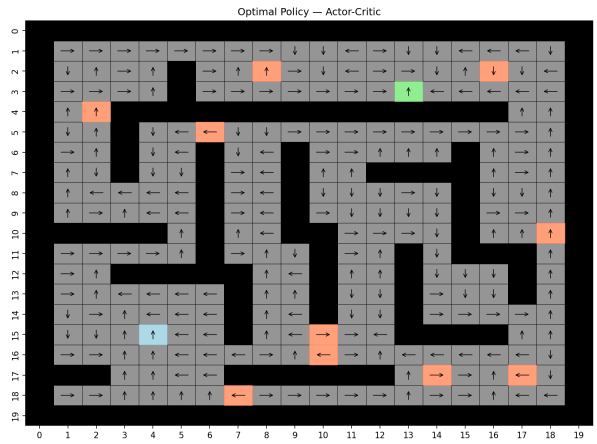

Figure 8: Optimal policy — Actor-Critic.

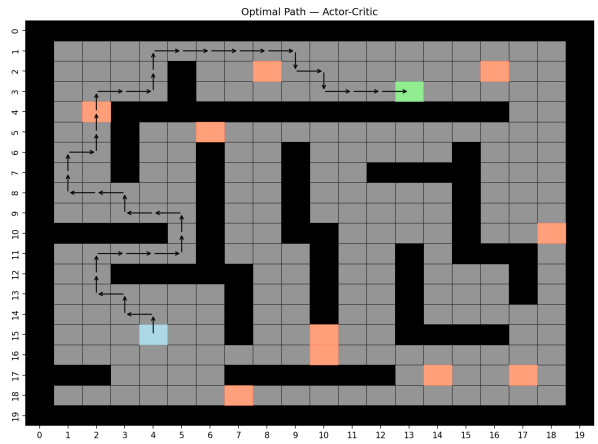

Figure 9: Optimal path — Actor-Critic.

Discussion: The Actor-Critic policy (Figure 8) is shown from one of the two successful runs ($\beta = 0.2$, 2,000 episodes). The arrows are overall directionally correct in regions the agent visited frequently during training, but some peripheral states show inconsistent directions due to sparse exploration under the softmax policy. The corresponding path (Figure 9) reaches the goal, though the trajectory is less direct compared to Q-Learning and SARSA, reflecting the higher variance inherent in policy-gradient-based methods.

4 Q3: Average Accumulated Reward per Algorithm

For each algorithm, the total reward per episode is averaged over 10 independent runs and plotted against the episode number. A rolling average (window = 50) is applied for smoother visualization.

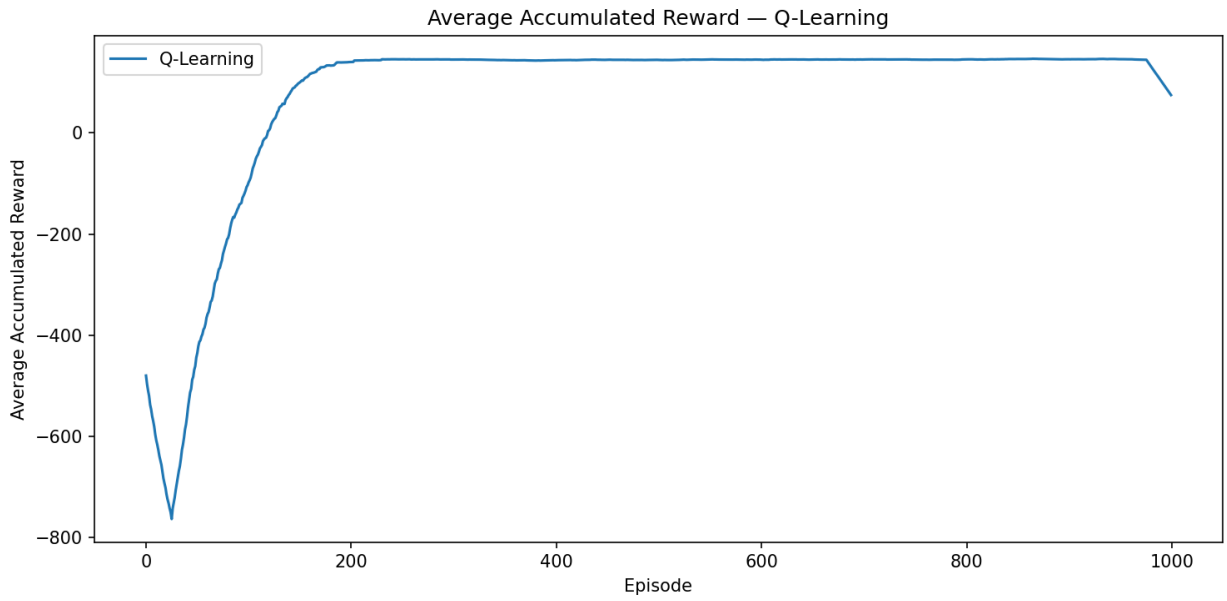

Figure 10: Average accumulated reward — Q-Learning.

Q-Learning (Figure 10): The reward curve rises steeply from around −800 in early episodes to near 0 by episode 200, then plateaus and stabilizes for the remainder of training. The sharp initial drop reflects the agent exhausting all 1,000 steps without reaching the goal in early episodes, accumulating large negative rewards from the −1 per-step cost. The rapid recovery indicates that Q-Learning's off-policy max-update efficiently propagates the +200 goal reward backward, allowing the agent to find shorter paths to the goal quickly.

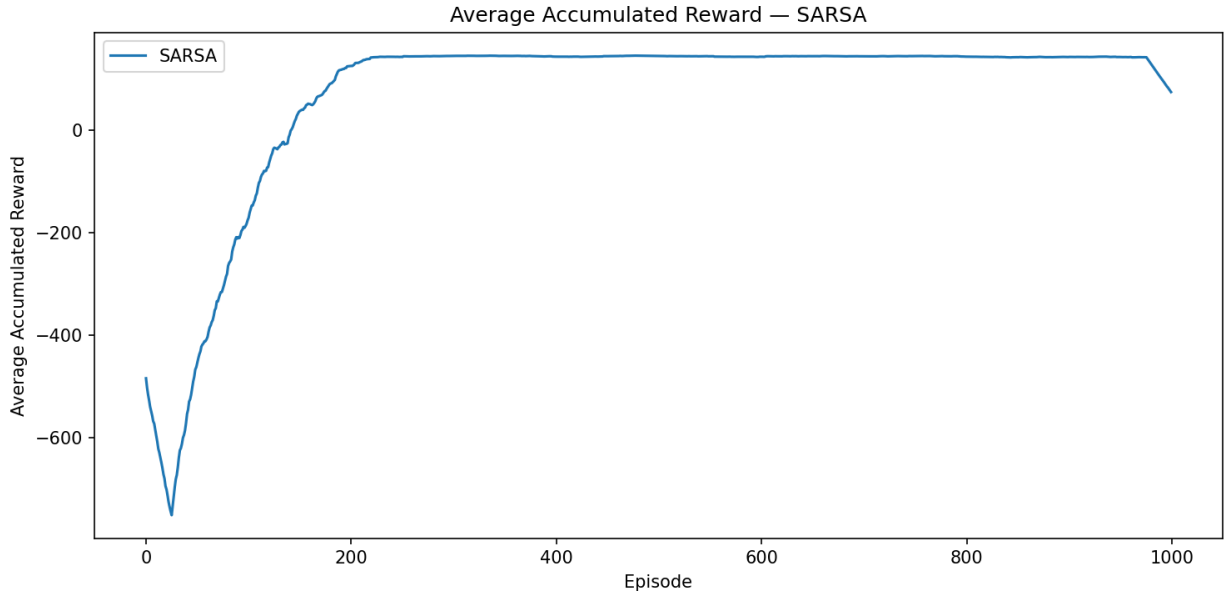

Figure 11: Average accumulated reward — SARSA.

SARSA (Figure 11): The SARSA reward curve follows a similar shape to Q-Learning, ris-

ing from approximately -600 to near 0 and plateauing around episode 300. The slightly slower rise compared to Q-Learning is consistent with on-policy learning: SARSA's target $r + \gamma Q(s', a')$ accounts for ε-greedy exploration noise, producing more conservative Q-value estimates that take longer to propagate the goal reward across the state space.

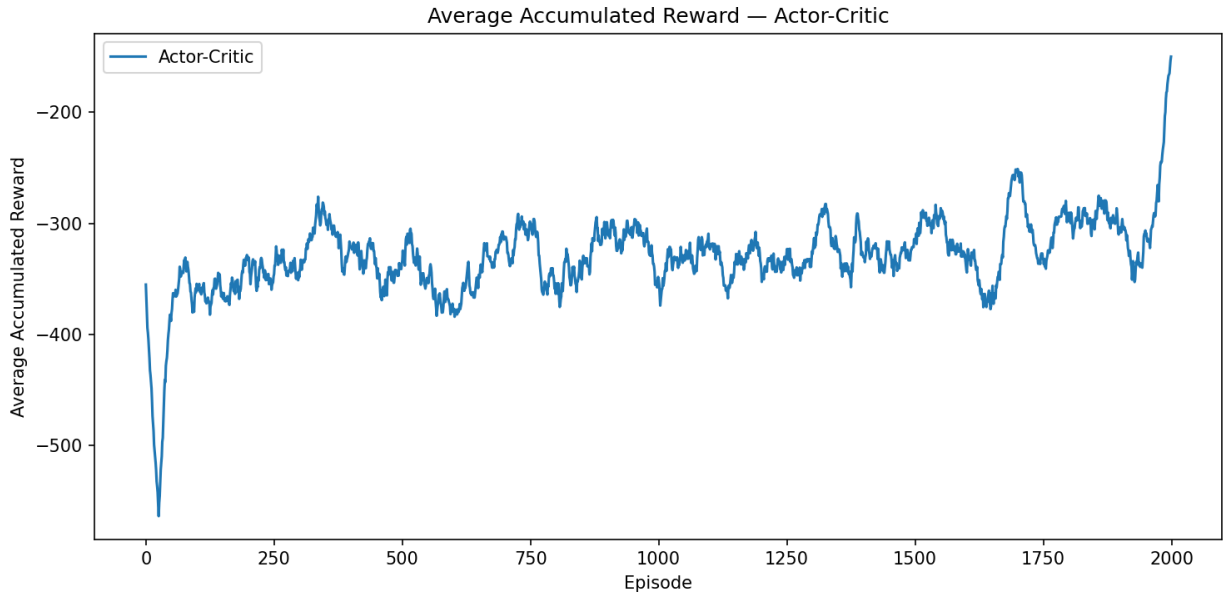

Figure 12: Average accumulated reward — Actor-Critic.

Actor-Critic (Figure 12): Unlike Q-Learning and SARSA, the Actor-Critic reward curve remains highly volatile throughout all 2,000 episodes, fluctuating between approximately -500 and -200 without a clear upward trend. This reflects the fundamental instability of the algorithm: the actor update $H(s, a) \leftarrow H(s, a) + \beta\, \delta\, (1 - \pi(a|s))$ depends on the accuracy of the TD error δ, which itself depends on a well-trained critic V. When this coordination fails, the policy oscillates rather than converging, which explains the noisy curve and the low success rate of 2/10.

5 Q4: Algorithm Comparison

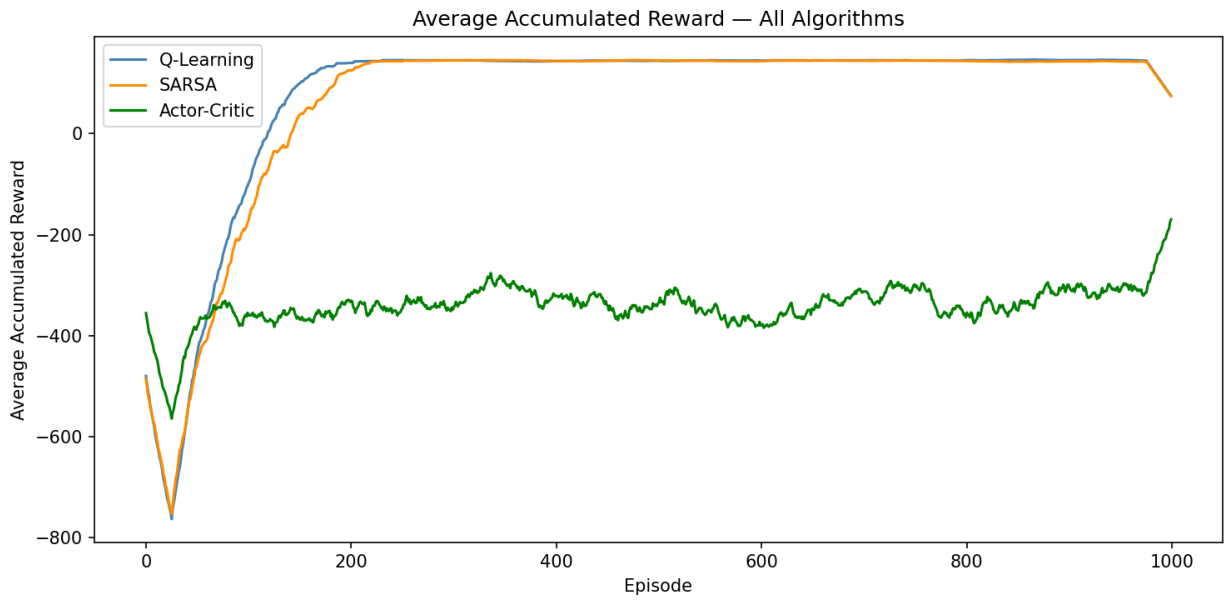

Figure 13: Comparison of average accumulated reward across all three algorithms.

Findings: Figure 13 reveals three distinct learning behaviors. Q-Learning (blue) converges the fastest, crossing zero reward by approximately episode 100 and reaching a final plateau near +100. SARSA (orange) follows a nearly identical trajectory but lags Q-Learning by roughly 10–20 episodes, consistent with the convergence episode means of 109.9 vs. 120.9 reported in Q5. Both Q-value methods ultimately achieve similar final performance, confirming that off-policy and on-policy TD learning are comparably effective on this maze once convergence is reached.

Actor-Critic (green) exhibits a fundamentally different behavior: the curve remains flat and highly noisy throughout all 1,000 episodes, never rising above −200. The gap between Actor-Critic and the Q-value methods widens after episode 100, precisely when Q-Learning and SARSA have already found the goal and begun accumulating positive rewards. This contrast highlights the key weakness of policy-gradient methods in sparse-reward environments: without a reliable critic $V(s)$, the TD error $\delta = r + \gamma V(s') - V(s)$ remains uninformative, and the actor cannot steer the softmax policy away from near-uniform exploration.

6 Q5: Convergence Speed Comparison

We track the first episode during training at which the greedy policy produces a valid path from start to goal. If the path is never found, the episode is recorded as the total number of training episodes (1,000 for Q-Learning and SARSA; 2,000 for Actor-Critic).

Algorithm	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	Mean
Q-Learning	115	120	105	111	123	111	127	74	108	105	109.9
SARSA	136	137	128	106	131	91	124	120	109	127	120.9
Actor-Critic*	107	2000	2000	134	2000	2000	144	2000	2000	2000	1438.5

Table 3: First episode where the greedy policy finds a valid start-to-goal path.

**Actor-Critic was trained for 2,000 episodes with $\beta = 0.2$. A value of 2,000 indicates the path was never found during training.*

Discussion: Q-Learning converges the fastest with a mean of 109.9 episodes, followed by SARSA at 120.9 episodes. The small difference of ∼11 episodes reflects Q-Learning's more aggressive off-policy update. Actor-Critic shows a bimodal pattern: the three runs that converged did so quickly (107, 134, 144 episodes), while the remaining seven never converged within 2,000 episodes, yielding a mean of 1,438.5. This confirms that Actor-Critic either converges rapidly when the critic and actor align early in training, or fails entirely due to the chicken-and-egg dependency between $V(s)$ and $\pi(a|s)$.

7 Q6: Effect of Learning Rate α

We select **Q-Learning** and vary $\alpha \in \{0.05, 0.1, 0.25, 0.5\}$ while keeping all other parameters fixed.

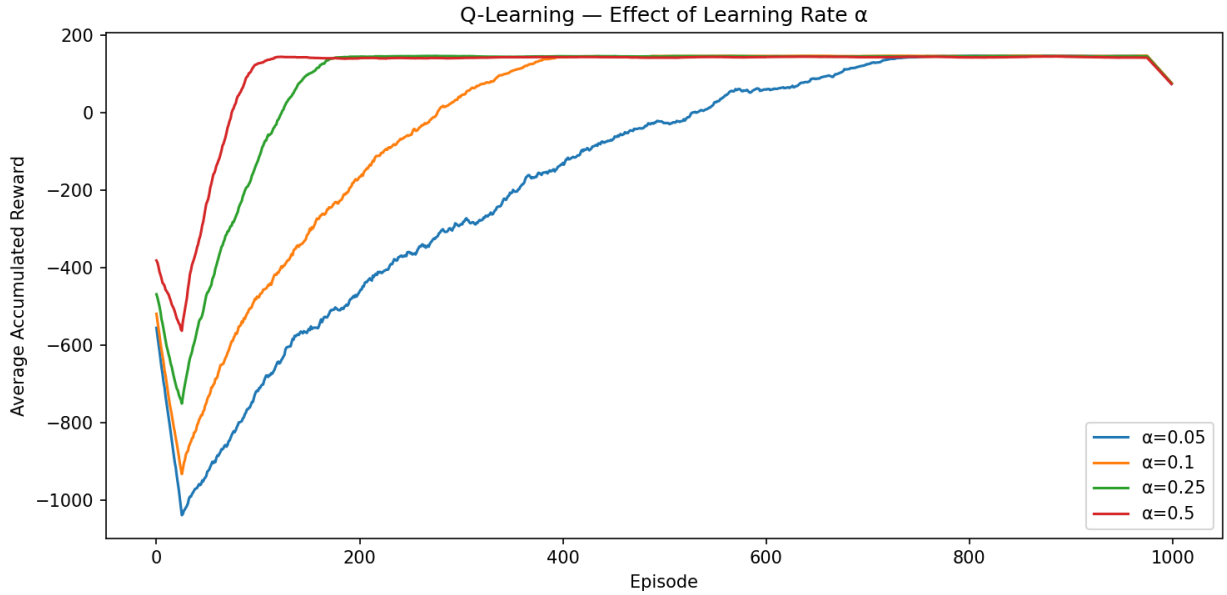

Figure 14: Effect of learning rate α on Q-Learning performance.

Discussion: Figure 14 shows a clear trade-off between convergence speed and early stability as α increases. With $\alpha = 0.5$ (red), the curve rises the fastest, crossing zero reward by approximately episode 50 and plateauing near +150 earliest among all settings; however, it also shows the steepest initial dip (≈ -450), indicating that large step sizes cause the Q-values to overreact to individual stochastic transitions in early episodes. With $\alpha = 0.25$ (green), convergence is nearly as fast but with a shallower initial dip, confirming it as a good balance between speed and stability. With

$\alpha = 0.1$ (orange), the curve rises more gradually, reaching the plateau around episode 300, while $\alpha = 0.05$ (blue) is the slowest, still climbing at episode 600 and not fully plateauing within 1,000 episodes.

In summary, larger α accelerates learning by weighting new experience more heavily in the update $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$, but at the cost of greater variance in early episodes. Smaller α produces smoother but slower convergence. For this maze environment, $\alpha = 0.25$ offers the best balance and was therefore selected as the default parameter.

Problem 2

8 Problem Setup and Parameters

We consider the 4-gene Boolean network (p53-Mdm2 feedback loop) from Project 2 (Problem 2). The state space consists of $2^4 = 16$ binary states $s \in \{0,1\}^4$, and the action space is $\mathcal{A} = \{a_1, a_2, a_3, a_4\}$ where $a_1 = [0,0,0,0]^T$, $a_2 = [0,1,0,0]^T$, $a_3 = [0,0,1,0]^T$, $a_4 = [0,0,0,1]^T$. The state transitions follow:

$$s_k = \overline{C\, s_{k-1} \oplus a_{k-1} \oplus n_k}$$

where C is the connectivity matrix, $\overline{(\cdot)}$ maps positive values to 1 and non-positive to 0, $\oplus$ is component-wise XOR, and n_k is i.i.d. Bernoulli noise with parameter p.

The reward function is $R(s,a,s') = 5\,s'_1 + 5\,s'_2 + 5\,s'_3 + 5\,s'_4 - c(a)$, where $c(a_1) = c(a_4) = 0$ and $c(a_2) = c(a_3) = 1$. The goal is to keep all genes ON (state $[1,1,1,1]$).

Four TD algorithms are implemented: **Q-Learning**, **SARSA**, **SARSA(λ)** ($\lambda = 0.95$), and **Tabular Actor-Critic**.

Parameter	Value
Noise parameter p	0.1
Discount factor γ	0.9
Learning rate α	0.25
Exploration rate ϵ	0.15 (Q-Learning, SARSA, SARSA(λ))
Actor learning rate β	0.05 (Actor-Critic)
Eligibility trace λ	0.95 (SARSA(λ))
Number of episodes	1,000
Max steps per episode	100
Number of independent runs	10

Table 4: Hyperparameters for Problem 2.

The key algorithmic addition in Problem 2 is **SARSA(λ)**, which extends SARSA with eligibility traces. At each step, the eligibility trace $e(s,a)$ is incremented for the visited state-action pair and decayed for all others:

$$\begin{aligned}
\delta &= r + \gamma Q(s',a') - Q(s,a), \\
e(s,a) &\leftarrow e(s,a) + 1, \\
Q(s,a) &\leftarrow Q(s,a) + \alpha\,\delta\,e(s,a), \quad \forall\, s,a, \\
e(s,a) &\leftarrow \gamma\,\lambda\,e(s,a), \quad \forall\, s,a.
\end{aligned}$$

This allows a single reward signal to update *all* recently visited state-action pairs, not just the current one.

9 Q1: Optimal Policies (10 Independent Runs)

For each algorithm, we report the learned greedy policy across all 10 runs. Each policy is a vector of length 16, specifying the best action (a_1, a_2, a_3, or a_4) for each of the 16 states. Although the problem statement lists a_1, a_2, a_3 as example actions, the full action space includes $a_4 = [0,0,0,1]^T$, which appears in the learned policies for states such as $[1,0,0,0]$ and $[1,1,0,0]$.

9.1 Q-Learning

	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}	s_{11}	s_{12}	s_{13}	s_{14}	s_{15}	s_{16}
Run 1	a_2	a_4	a_3	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_2	a_4	a_2	a_2	a_2
Run 2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_3	a_1	a_2	a_2	a_2
Run 3	a_3	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_2	a_1	a_4	a_2	a_2
Run 4	a_2	a_2	a_3	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_2	a_4	a_2	a_2	a_2
Run 5	a_2	a_3	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_3	a_4	a_2	a_2	a_2
Run 6	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_1	a_2	a_2	a_2	a_4	a_2	a_2	a_2
Run 7	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_2	a_4	a_2	a_2	a_2
Run 8	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_3	a_4	a_4	a_2	a_2
Run 9	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_3	a_2	a_4	a_2	a_2	a_2
Run 10	a_3	a_3	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_3	a_2	a_2	a_4	a_2	a_2	a_2

Table 5: Q-Learning optimal policies across 10 runs.

9.2 SARSA

	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}	s_{11}	s_{12}	s_{13}	s_{14}	s_{15}	s_{16}
Run 1	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_1	a_2	a_2	a_3	a_4	a_2	a_2	a_2
Run 2	a_2	a_3	a_3	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_2	a_4	a_2	a_2	a_2
Run 3	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_2	a_4	a_2	a_2	a_2
Run 4	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_1	a_2	a_4	a_2	a_4	a_2	a_2	a_2
Run 5	a_3	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_3	a_3	a_2	a_2	a_4	a_4	a_2	a_2
Run 6	a_2	a_2	a_2	a_2	a_2	a_1	a_2	a_2	a_1	a_2	a_4	a_2	a_4	a_2	a_2	a_2
Run 7	a_2	a_1	a_3	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_2	a_4	a_2	a_2	a_2
Run 8	a_3	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_3	a_2	a_4	a_2	a_4	a_2	a_2	a_2
Run 9	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_3	a_2	a_2	a_4	a_4	a_2	a_2
Run 10	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_3	a_2	a_2	a_3	a_4	a_2	a_2	a_2

Table 6: SARSA optimal policies across 10 runs.

9.3 SARSA(λ)

	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}	s_{11}	s_{12}	s_{13}	s_{14}	s_{15}	s_{16}
Run 1	a_2	a_2	a_3	a_3	a_1	a_2	a_2	a_4	a_4	a_3	a_2	a_3	a_4	a_2	a_4	a_1
Run 2	a_3	a_3	a_2	a_2	a_2	a_2	a_2	a_1	a_3	a_4	a_3	a_3	a_4	a_2	a_2	a_2
Run 3	a_4	a_2	a_1	a_4	a_2	a_2	a_2	a_3	a_3	a_2	a_4	a_2	a_4	a_2	a_1	a_2
Run 4	a_3	a_1	a_3	a_2	a_3	a_1	a_2	a_2	a_1	a_2	a_2	a_2	a_4	a_2	a_2	a_2
Run 5	a_2	a_2	a_2	a_3	a_2	a_2	a_1	a_4	a_3	a_4	a_3	a_3	a_4	a_2	a_1	a_2
Run 6	a_3	a_2	a_2	a_2	a_2	a_4	a_2	a_1	a_1	a_2	a_2	a_2	a_4	a_4	a_2	a_2
Run 7	a_4	a_2	a_3	a_2	a_2	a_2	a_2	a_1	a_3	a_3	a_2	a_2	a_4	a_2	a_2	a_2
Run 8	a_2	a_2	a_3	a_2	a_2	a_2	a_1	a_2	a_3	a_3	a_1	a_2	a_1	a_2	a_2	a_2
Run 9	a_2	a_2	a_1	a_2	a_2	a_2	a_2	a_2	a_3	a_2	a_2	a_2	a_1	a_2	a_2	a_2
Run 10	a_2	a_2	a_2	a_4	a_2	a_2	a_4	a_1	a_4	a_4	a_2	a_3	a_1	a_2	a_2	a_2

Table 7: SARSA(λ) optimal policies across 10 runs ($\lambda = 0.95$).

9.4 Actor-Critic

	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}	s_{11}	s_{12}	s_{13}	s_{14}	s_{15}	s_{16}
Run 1	a_3	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_3	a_2	a_1	a_2	a_2	a_2
Run 2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_3	a_2	a_3	a_2	a_4	a_2	a_2	a_2
Run 3	a_3	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_3	a_3	a_2	a_3	a_4	a_2	a_2	a_2
Run 4	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_2	a_4	a_2	a_2	a_2
Run 5	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_3	a_3	a_3	a_4	a_4	a_2	a_2
Run 6	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_1	a_2	a_2	a_2	a_4	a_4	a_2	a_2
Run 7	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_3	a_4	a_2	a_2	a_2
Run 8	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_3	a_3	a_2	a_2	a_4	a_2	a_2	a_2
Run 9	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_4	a_2	a_2	a_2	a_4	a_2	a_2	a_2
Run 10	a_3	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_3	a_2	a_2	a_2	a_4	a_2	a_2	a_2

Table 8: Actor-Critic optimal policies across 10 runs.

Key code — SARSA(λ) update with eligibility traces:

```
# TD error
delta = reward + gamma * Q[next_state, next_action] - Q[state, action]

# Accumulate trace for visited (s, a)
e[state, action] += 1.0

# Update Q and decay traces for all (s, a)
Q += alpha * delta * e
e *= gamma * lam
```

Figure 15: SARSA(λ) core update implementation.

Discussion: The policies learned by Q-Learning, SARSA, and Actor-Critic are highly consistent across 10 runs: most states converge to the same action in all runs. Notably, state $[1, 1, 1, 1]$ consistently selects a_2 across all three algorithms, and state $[1, 1, 0, 0]$ consistently selects a_4 (a zero-cost action), reflecting that the algorithms have learned to prefer free interventions when possible. In contrast, SARSA(λ) with $\lambda = 0.95$ shows substantially more variation across runs — different runs often select different actions for the same state. This is because the long eligibility traces assign credit to many preceding state-action pairs simultaneously, introducing high variance in Q-value updates for rarely visited states where the trace-weighted updates can be noisy and inconsistent across independent runs.

10 Q2: Average Accumulated Reward per Algorithm

For each algorithm, the total reward per episode is averaged over 10 independent runs.

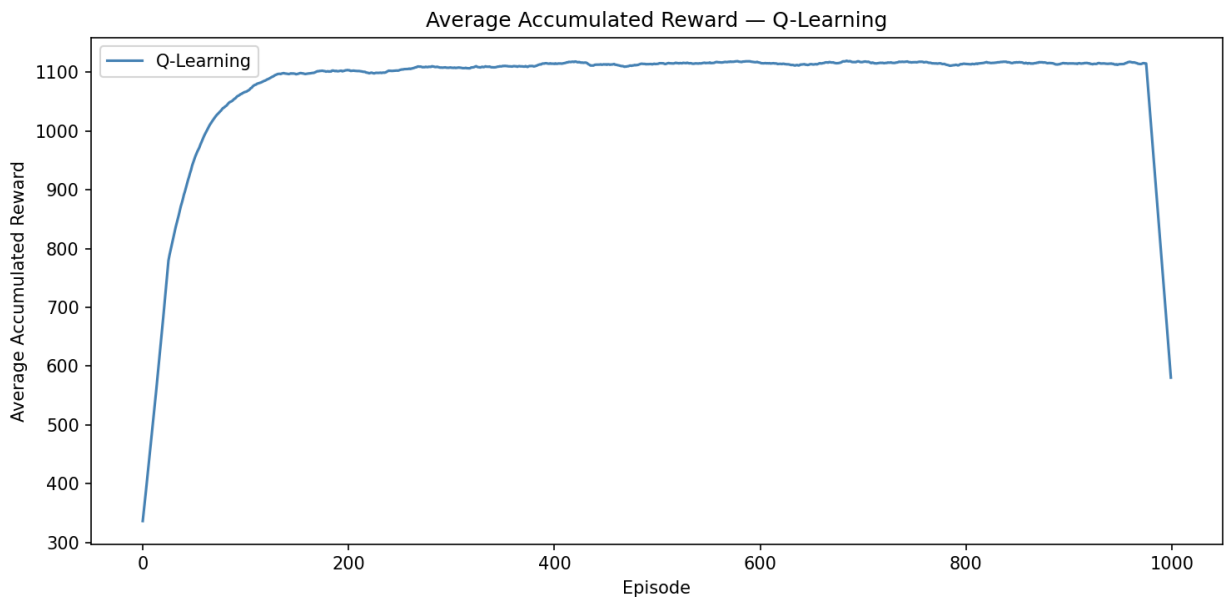

Figure 16: Average accumulated reward — Q-Learning (Problem 2).

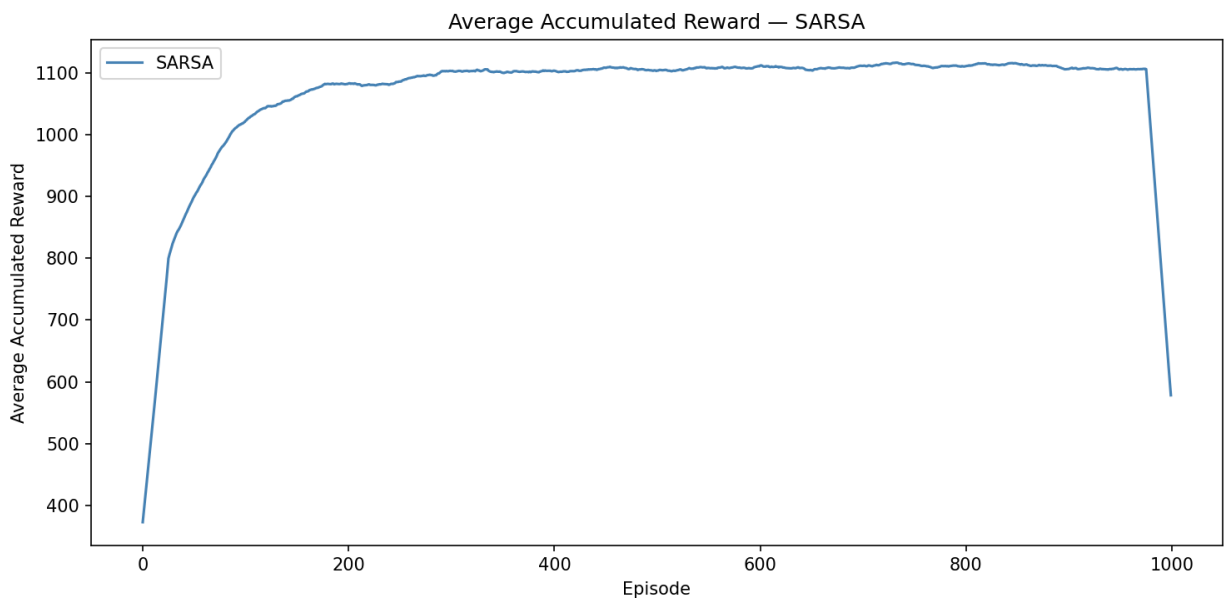

Figure 17: Average accumulated reward — SARSA (Problem 2).

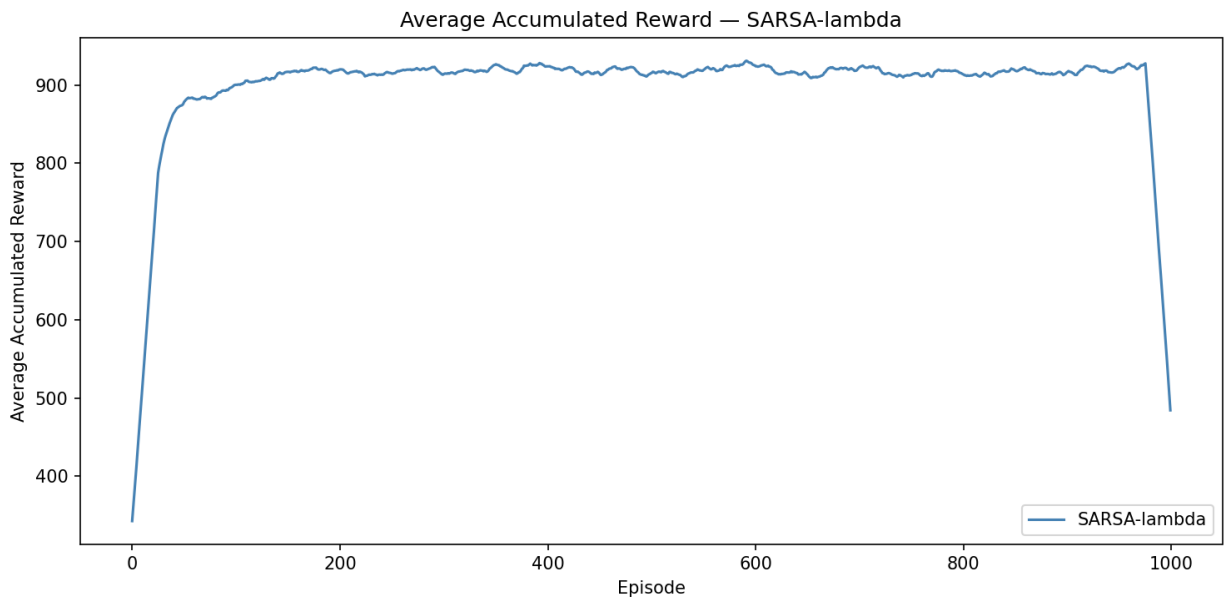

Figure 18: Average accumulated reward — SARSA(λ) (Problem 2).

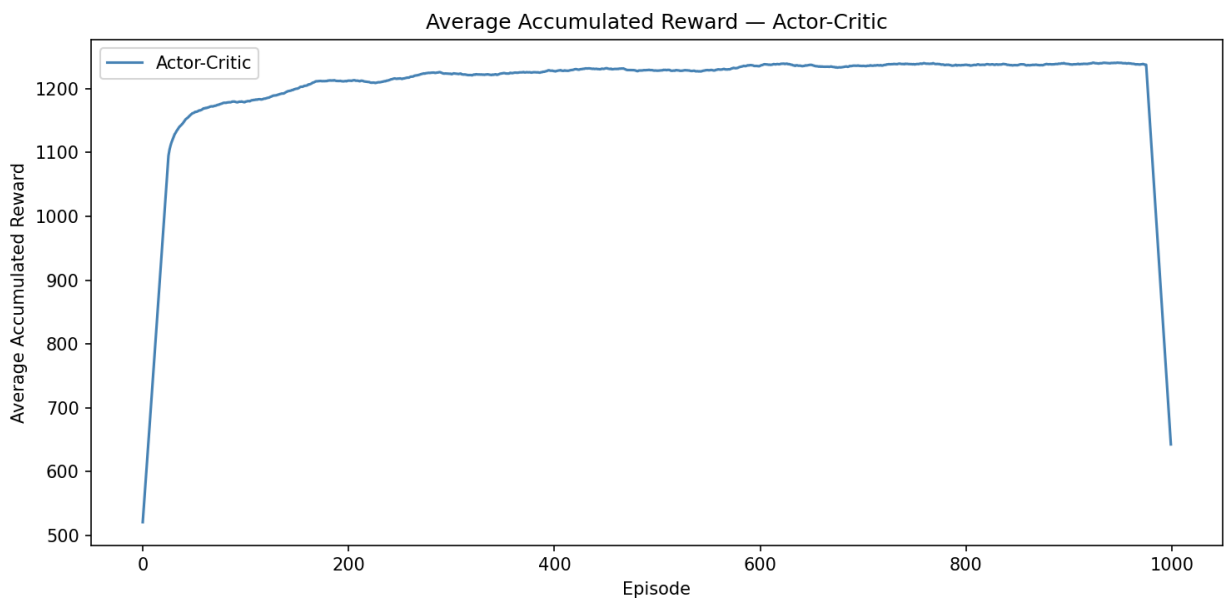

Figure 19: Average accumulated reward — Actor-Critic (Problem 2).

Discussion: All four algorithms show a rapid rise in average accumulated reward during the first 50–100 episodes, reflecting the small state space (only 16 states) which allows Q-values to propagate quickly. Q-Learning (Figure 16) and SARSA (Figure 17) both converge to a plateau near 1,100 by approximately episode 100, confirming reliable and stable learning. Actor-Critic (Figure 19) converges fastest and reaches the highest plateau of approximately 1,230, because the softmax policy continuously explores all actions with non-zero probability, allowing the critic $V(s)$ to receive gradient signals from all states more uniformly than ε-greedy methods.

SARSA(λ) (Figure 18) plateaus at a noticeably lower value of approximately 910, despite converging at a similar speed. This is because the long eligibility traces ($\lambda = 0.95$, decay factor $\gamma\lambda = 0.855$) propagate TD errors back to many preceding state-action pairs simultaneously. With only 100 steps per episode and a small state space, this causes over-propagation: states that were visited early in the episode receive large updates even when they are far from the high-reward region $[1, 1, 1, 1]$, introducing noise that depresses the average reward.

All four curves show a sharp drop near episode 1,000, which is an artifact of the rolling-mean smoothing window extending beyond the end of the data.

11 Q3: Algorithm Comparison

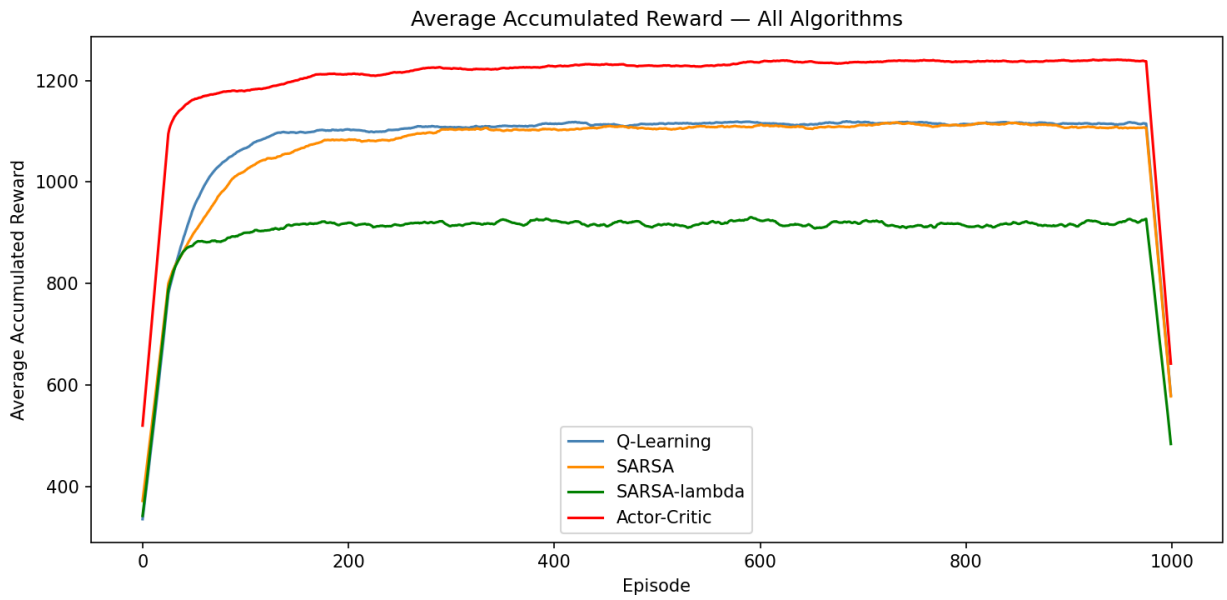

Figure 20: Comparison of average accumulated reward across all four algorithms (Problem 2).

Findings: Figure 20 reveals a clear performance ranking across the four algorithms. Actor-Critic (red) achieves the highest plateau at approximately 1,230, rising steeply within the first 20 episodes and maintaining stable performance throughout. Q-Learning (blue) and SARSA (orange) converge to nearly identical plateaus near 1,100, with Q-Learning rising slightly faster in episodes 0–100 due to its off-policy max-update. The small gap between Q-Learning and SARSA is consistent with their difference in update targets: Q-Learning bootstraps toward $\max_{a'} Q(s', a')$ while SARSA uses the actual ε-greedy action value, leading to marginally more conservative estimates.

SARSA(λ) (green) is the clear outlier, stabilizing at approximately 910 — roughly 200 points below Q-Learning and SARSA. As discussed in Q2, the long traces ($\lambda = 0.95$) over-propagate credit in this small environment, reducing rather than improving performance compared to standard SARSA. This result highlights that larger λ is not always beneficial: its advantage is most pronounced in environments with long delayed rewards, while in the gene network where the reward signal is dense (received at every step), long traces introduce unnecessary variance.

12 Q4: State Visitation Counts

After training, we execute each algorithm's greedy policy for 100 episodes and record how many times each of the 16 states is visited.

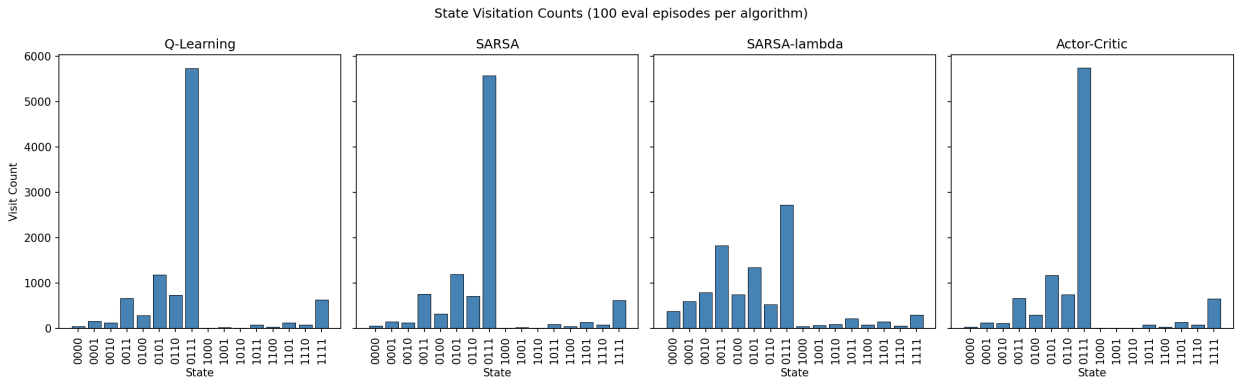

Figure 21: State visitation counts (100 greedy episodes).

Discussion: Q-Learning, SARSA, and Actor-Critic all show highly concentrated visitation on state $[0, 1, 1, 1]$ (approximately 5,700 visits out of 10,000 total steps), with state $[1, 1, 1, 1]$ receiving around 640 visits. This pattern reveals that the learned policies drive the gene network toward the partially activated state $[0, 1, 1, 1]$ and keep it there, rather than reaching the fully activated $[1, 1, 1, 1]$. This is consistent with the reward structure: the Bernoulli noise ($p = 0.1$) stochastically flips genes, making it difficult to maintain all four genes ON simultaneously, so the policy settles for the next best stable attractor.

SARSA(λ) shows a markedly different and more dispersed visitation pattern: state $[0, 1, 1, 1]$ receives only 2,725 visits, while states such as $[0, 0, 1, 1]$ (1,835) and $[0, 1, 0, 1]$ (1,347) each receive substantially more visits than under the other algorithms. This confirms that SARSA(λ)'s policy is of lower quality — it fails to consistently steer the system toward high-reward states, consistent with its lower average reward of $\sim$910 observed in Q2 and Q3.

13 Q5: Effect of λ on SARSA(λ)

We repeat the SARSA(λ) experiment with $\lambda \in \{0, 0.5, 0.95\}$, where $\lambda = 0$ reduces to standard one-step SARSA.

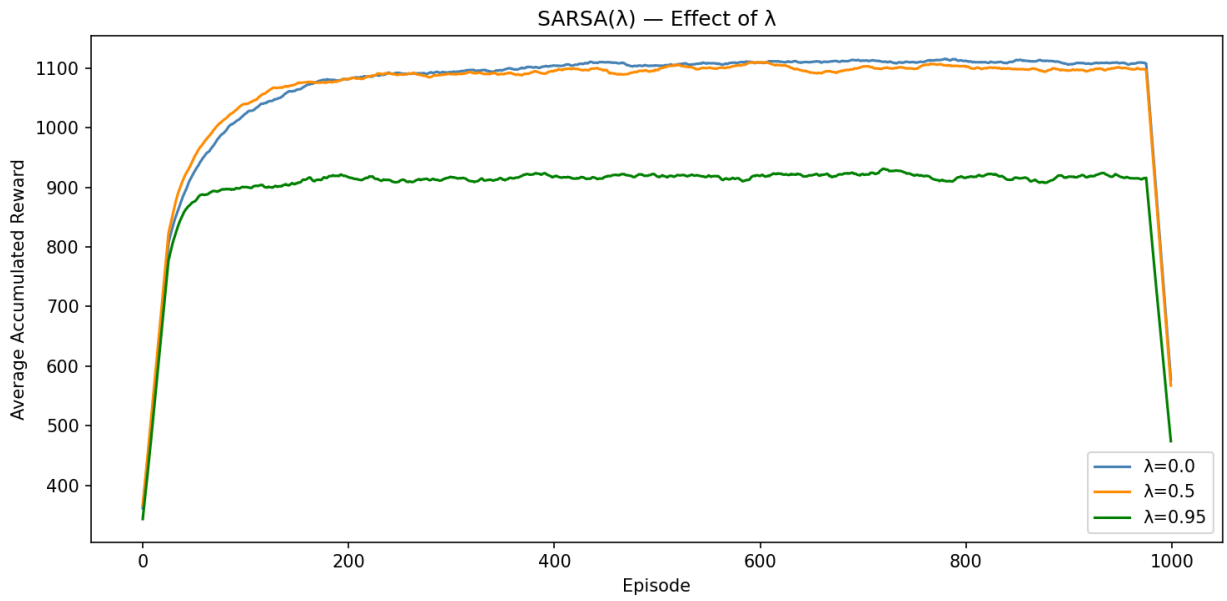

Figure 22: Effect of λ on SARSA(λ) performance.

Discussion: Figure 22 shows that $\lambda = 0$ (standard one-step SARSA) achieves the highest average accumulated reward, converging to approximately 1,080 within the first 100 episodes and maintaining a stable plateau. $\lambda = 0.5$ produces intermediate performance, converging to a slightly lower plateau, while $\lambda = 0.95$ yields the lowest reward of approximately 910 throughout training.

This counterintuitive result — where longer traces hurt performance — can be explained by the properties of this environment. With a maximum episode length of only 100 steps and a dense reward signal (received at every step), eligibility traces with large λ propagate credit too far back in time. The effective decay factor $\gamma\lambda = 0.9 \times 0.95 = 0.855$ means that state-action pairs visited 10 steps ago still receive 85% of the current TD error, causing noisy and inconsistent updates for states that were visited under different conditions. In contrast, $\lambda = 0$ assigns credit only to the immediately preceding state-action pair, which is more reliable in a dense-reward setting.